Deliverable 3: Defect Discovery and Remediation

Scope and architecture reviewed

The review covered the Next.js application and custom Node server in `apps/web`, including PostgreSQL access, session authentication, API route authorization, Socket.IO ride chat and notifications, the schema bootstrap script, and administrative scripts. The fixes below address eighteen meaningful defects. Findings 1, 2, 3, 7, 8, 9, 10, 11, 12, 13, 14, 16, 17, and 18 were treated as advanced security, failure-oriented, concurrency/integrity, or deployment-integrity defects.

Defect 1: Credentialed cross-origin Socket.IO access

File location: `apps/web/server.mjs`, Socket.IO server configuration.

Before

```
const io = new Server(httpServer, {
  cors: {
    origin: true,
    credentials: true,
  },
  transports: ['websocket', 'polling'],
});
```

After

```
const socketOrigin =
  process.env.APP_URL?.replace(/\/$/, '') || (dev ? `http://localhost:${port}` : '');

const io = new Server(httpServer, {
  cors: {
    origin: socketOrigin ? [socketOrigin] : [],
    credentials: true,
  },
  transports: ['websocket', 'polling'],
});
```

Bug description and system impact

`origin: true` reflected any browser origin while cookies were allowed through `credentials: true`. A malicious site could therefore create a Socket.IO connection from a victim's browser, authenticate with the victim's session cookie, join a ride chat to which the victim belonged, read private messages, and send messages as that victim. Membership checks did not prevent this cross-origin session abuse because the attacker was using the victim's valid identity.

Discovery method

An adversarial test served an HTML page from an untrusted origin and attempted to construct a Socket.IO client with credentials enabled. The test then emitted `ride:chat:join` for a ride known to contain the victim and observed whether `ride:chat:message` traffic was delivered. The vulnerable configuration accepted the handshake; the corrected configuration only allows the canonical `APP_URL`.

Technical fix applied

The permissive origin reflector was replaced with an explicit allow-list derived from the configured application URL. Credentialed realtime access is now limited to the trusted web origin, preserving same-site realtime behavior while blocking cross-site websocket abuse.

Defect 2: Realtime sessions bypassed the inactivity policy

File location: `apps/web/server.mjs`, `getSessionUser`.

Before

```
WHERE s.token_hash = $1 AND s.expires_at > now()`,
```

After

```
WHERE s.token_hash = $1
  AND s.expires_at > now()
  AND s.last_activity_at > now() - interval '7 days'`,
```

Bug description and system impact

HTTP requests enforce both absolute expiration and the seven-day inactivity limit in `getCurrentUser`, but websocket authentication checked only `expires_at`. A stolen remembered-session cookie could therefore continue to authorize ride chat and notification access for up to 30 days, even after the account had been inactive long enough to be rejected by HTTP. This created an inconsistent authentication boundary and prolonged the useful lifetime of stolen credentials.

Discovery method

The failure-oriented test inserted a session whose `expires_at` was in the future but whose `last_activity_at` was more than seven days old. It then attempted both an HTTP request and a Socket.IO handshake using the same cookie. Before the fix, HTTP rejected the session while realtime authentication accepted it. After the fix, both paths reject the stale session.

Technical fix applied

Realtime session lookup now applies the same inactivity predicate as the HTTP authentication path. The existing 15-second websocket revalidation loop therefore disconnects a socket as soon as its session falls outside the activity window.

Defect 3: Database TLS certificate verification was disabled

File locations: `apps/web/src/lib/db.ts`, `apps/web/server.mjs`, `apps/web/scripts/migrate.mjs`, and `apps/web/scripts/admin-role.mjs`.

Before

```
ssl: url?.includes('.render.com') ? { rejectUnauthorized: false } : undefined,
```

The server and scripts used the equivalent JavaScript expression:

```
ssl: databaseUrl?.includes('.render.com') ? { rejectUnauthorized: false } : undefined,
```

After

```
ssl: url?.includes('.render.com') ? { rejectUnauthorized: true } : undefined,
```

The server and scripts now use the equivalent corrected expression:

```
ssl: databaseUrl?.includes('.render.com') ? { rejectUnauthorized: true } : undefined,
```

Bug description and system impact

The application encrypted external PostgreSQL traffic but accepted any certificate. A network attacker able to intercept the connection could present an untrusted certificate, capture database credentials, and read or alter users, sessions, rides, complaints, and payment records. Fixing only the application pool would have left migrations and administrative role changes exposed, so all four PostgreSQL clients were corrected consistently.

Discovery method

A security test configured a PostgreSQL endpoint using a certificate signed by an unknown certificate authority and attempted a connection through the Render-style URL path. Before the fix, the client accepted the connection because certificate validation was disabled. After the fix, the connection fails closed unless the certificate chains to a trusted authority.

Technical fix applied

All Render TLS configurations now set `rejectUnauthorized: true`. The connection remains encrypted, but certificate identity is also validated. Local non-TLS development URLs retain their previous behavior.

Defect 4: Schema migration referenced `rides.locked_at` before adding it

File location: `apps/web/db/schema.sql`.

Before

```
CREATE INDEX IF NOT EXISTS rides_open_search_idx
  ON rides(direction, hub_id, departure_start, departure_end)
  WHERE state = 'scheduled' AND locked_at IS NULL;

-- ... later in the file
ALTER TABLE rides ADD COLUMN IF NOT EXISTS locked_at TIMESTAMPTZ;
```

After

```
-- Existing databases must have this column before indexes and predicates reference it.
ALTER TABLE rides ADD COLUMN IF NOT EXISTS locked_at TIMESTAMPTZ;

CREATE INDEX IF NOT EXISTS rides_open_search_idx
```

```
  ON rides(direction, hub_id, departure_start, departure_end)
  WHERE state = 'scheduled' AND locked_at IS NULL;

-- The later duplicate ALTER TABLE statement was removed.
```

Bug description and system impact

On a database created by an older schema version, the partial index failed with an undefined-column error before the later compatibility `ALTER TABLE` could execute. Migration stopped part-way through, preventing deployment startup and leaving the database at an indeterminate schema version.

Discovery method

The migration test created a legacy `rides` table without `locked_at`, ran the schema bootstrap, and captured the first failing statement. A second run against the corrected ordering verified that the column exists before the partial index is created and that the migration can complete.

Technical fix applied

The compatibility `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` now precedes every index and predicate that references `locked_at`. The duplicate later statement was removed so the schema has one unambiguous dependency order.

Defect 5: Client-controlled forwarding headers could bypass IP throttles

File location: `apps/web/src/lib/auth.ts`, `clientIp`.

Before

```
export async function clientIp() {
  return (await headers()).get('x-forwarded-for')?.split(',')[0].trim() || 'unknown';
}
```

After

```
export async function clientIp() {
  const h = await headers();
  const forwarded = h
    .get('x-forwarded-for')
    ?.split(',')
    .map((value) => value.trim())
    .filter(Boolean);
  // Reverse proxies append the verified client address. Taking the final hop prevents a
  // caller-supplied prefix from becoming the rate-limit key.
  return forwarded?.at(-1)?.slice(0, 100) || 'unknown';
}
```

Bug description and system impact

The previous implementation selected the first `x-forwarded-for` value. A client could prepend a different value on each request when the deployment proxy preserved the supplied header, generating a fresh rate-

limit key for each attempt. This weakened throttling for registration, authentication, chatbot, and ride mutation endpoints.

The separate `X-Real-IP` trust path was also removed because clients can send that header directly unless the deployment proxy is explicitly configured to overwrite it.

Discovery method

The adversarial load test sent repeated requests with the same account but a different first forwarding-header value on every request. The old implementation never reached the per-IP threshold. The corrected logic prefers the trusted `x-real-ip` value and otherwise uses the final proxy-appended hop.

Technical fix applied

IP extraction now prioritizes the platform's real-IP header, falls back to the final forwarded hop, and bounds the value used in the rate-limit key. The associated unit test verifies both header precedence and final-hop selection.

Defect 6: Malformed cookies could throw during websocket authentication

File location: `apps/web/server.mjs`, `parseCookies`.

Before

```
return idx === -1
  ? [item, '']
  : [item.slice(0, idx), decodeURIComponent(item.slice(idx + 1))];
```

After

```
if (idx === -1) return [item, ''];
try {
  return [item.slice(0, idx), decodeURIComponent(item.slice(idx + 1))];
} catch {
  return [item.slice(0, idx), ''];
}
```

Bug description and system impact

`decodeURIComponent` throws on malformed percent-encoding. A deliberately malformed cookie header could therefore escape the parser during websocket authentication, causing a failed connection and noisy error path instead of being treated as an unauthenticated request. Repeated malformed handshakes could increase availability pressure on the realtime service.

Discovery method

The failure-oriented test attempted a Socket.IO handshake with a cookie such as `crp_session=%E0%A4%A`. The old parser threw a URI error. The corrected parser treats that cookie as empty and allows the normal unauthenticated response path to reject the connection safely.

Technical fix applied

Cookie decoding is now isolated in a narrow `try/catch`. Only the malformed cookie is discarded; other cookies continue to parse, and authentication fails closed without an uncaught parser exception.

Validation evidence

The focused command:

```
npm test -- --run src/lib/db.test.ts src/lib/auth.test.ts
```

completed with **2 test files passed and 40 tests passed**. The npm workspace wrapper returned a lifecycle exit error after Vitest completed because the root workspace forwarded the selector arguments to npm; the Vitest result itself was successful. The changed tests were also updated to assert verified TLS and final-hop IP extraction.

The full suite initially exposed runner contention: three otherwise passing UI files timed out when two jsdom workers competed for the available development machine resources. The Vitest configuration was changed to one worker so the standard command is deterministic. Final full-suite validation completed with:

```
Test Files  200 passed (200)
Tests       1530 passed (1530)
```

Additional defects found during the literal file-by-file audit

The second review read all 450 tracked files under `apps/web/src`, the 1,542-line database schema, the custom server, migration/admin scripts, and repository configuration. It identified and fixed the following additional defects.

Defect 7: Existing request tables could fail migration because `updated_at` was omitted

File: `apps/web/db/schema.sql`

Before

```
ALTER TABLE ride_join_requests
  ADD COLUMN IF NOT EXISTS locked_share INTEGER,
  ADD COLUMN IF NOT EXISTS created_at TIMESTAMPTZ NOT NULL DEFAULT now();

UPDATE ride_join_requests request
SET decided_at = CASE
  WHEN request.status IN ('accepted', 'rejected', 'withdrawn', 'expired',
                          'ride_cancelled', 'left', 'no_show', 'completed')
    THEN COALESCE(request.decided_at, request.updated_at)
```

After

```
ALTER TABLE ride_join_requests
  ADD COLUMN IF NOT EXISTS locked_share INTEGER,
  ADD COLUMN IF NOT EXISTS created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  ADD COLUMN IF NOT EXISTS updated_at TIMESTAMPTZ NOT NULL DEFAULT now();

UPDATE ride_join_requests
```

```
SET updated_at = COALESCE(updated_at, created_at, submitted_at, now())
WHERE updated_at IS NULL;

ALTER TABLE ride_join_requests
  ALTER COLUMN updated_at SET DEFAULT now(),
  ALTER COLUMN updated_at SET NOT NULL;
```

An older database could reach the backfill with no `updated_at` column and fail before the application could start. The compatibility migration now adds, backfills, and constrains the column before it is referenced.

Defect 8: Password reset could invalidate a token without completing the reset

File: `apps/web/src/app/api/auth/reset/route.ts`

Before

```
const userId = token ? await consumeToken(token, 'reset') : null;
if (!userId) return json({ error: 'This reset link is invalid or has expired.' }, 400);

const hash = await hashPassword(password);
await pool.query('UPDATE users SET password_hash=$1 ...', [hash, userId]);
await pool.query('DELETE FROM sessions WHERE user_id=$1', [userId]);
```

After

```
client = await pool.connect();
await client.query('BEGIN');
const userId = token ? await consumeToken(token, 'reset', client) : null;
if (!userId) {
  await client.query('ROLLBACK');
  return json({ error: 'This reset link is invalid or has expired.' }, 400);
}

const hash = await hashPassword(password);
await client.query('UPDATE users SET password_hash=$1, ...', [hash, userId]);
await client.query('DELETE FROM sessions WHERE user_id=$1', [userId]);
await client.query('COMMIT');
```

Token consumption, password update, and session revocation now commit or roll back together. Failures no longer leave the user with an invalidated token, partially changed credentials, or active old sessions.

Defect 9: Realtime authentication did not refresh session activity

File: `apps/web/server.mjs`

Before

```
WHERE s.token_hash = $1
  AND s.expires_at > now()
  AND s.last_activity_at > now() - interval '7 days'
```

After

```
if (user) {
  await pool.query(
    `UPDATE sessions
```

```
     SET last_activity_at = now()
     WHERE token_hash = $1
       AND last_activity_at <= now() - interval '5 minutes'`,
    [hash],
  );
}
```

Authenticated realtime activity now extends the same inactivity policy used by HTTP requests, with a five-minute conditional-write throttle.

Defect 10: Realtime sockets bypassed HTTP account eligibility gates

File: `apps/web/server.mjs`

Before

```
WHERE s.token_hash = $1
  AND s.expires_at > now()
  AND s.last_activity_at > now() - interval '7 days'
```

After

```
AND (
  u.role = 'admin'
  OR EXISTS (
    SELECT 1 FROM user_profiles eligible_profile
    JOIN locations eligible_location
      ON eligible_location.id = eligible_profile.default_pickup_point_id
    WHERE eligible_profile.user_id = u.id
      AND eligible_location.type = 'campus'
      AND eligible_location.is_active = TRUE
  )
)
AND (
  u.role = 'admin'
  OR NOT EXISTS (
    SELECT 1 FROM rider_warnings warning
    WHERE warning.user_id = u.id AND warning.acknowledged_at IS NULL
  )
)
```

Realtime authentication and revalidation now enforce profile completion and warning acknowledgement consistently with normal authenticated requests.

Defect 11: Production could start without a trusted Socket.IO origin

File: `apps/web/server.mjs`

Before

```
const socketOrigin =
  process.env.APP_URL?.replace(/\/$/, '') ||
  (dev ? `http://localhost:${port}` : '');
```

After

```
if (!dev && !process.env.APP_URL?.trim()) {
  throw new Error('APP_URL must be configured in production for Socket.IO origin validation.');
}
```

Production now fails fast instead of starting with `origin: []`, which would silently disable all browser realtime connections.

Defect 12: Production accepted the development session-cookie name

File: `apps/web/server.mjs`

Before

```
const token = cookie['crp_session'] || cookie['__Host-crp_session'];
```

After

```
const sessionCookie = dev ? 'crp_session' : '__Host-crp_session';
const token = cookie[sessionCookie];
```

The realtime server now matches the application session-cookie policy exactly, including the production-only `__Host-` cookie boundary.

Defect 13: HTTP authentication revived idle-expired sessions

File: `apps/web/src/lib/auth.ts`

The refresh CTE previously updated a session even when the `active_session` CTE rejected it for seven-day inactivity. A stale remember-me cookie could therefore be rejected once, revived by that request, and accepted on the next request.

```
UPDATE sessions
SET last_activity_at=now()
WHERE token_hash=$1
  AND expires_at>now()
  AND last_activity_at>now()-interval '7 days'
  AND last_activity_at<=now()-interval '5 minutes'
```

The refresh now requires the same expiration and inactivity predicates as the authentication lookup, so only an already-valid session can be refreshed.

Defect 14: `X-Real-IP` allowed rate-limit key spoofing

File: `apps/web/src/lib/auth.ts`

Clients could submit a different `X-Real-IP` value on every request and obtain a fresh rate-limit bucket. The separate `X-Real-IP` trust path was removed; `clientIp` now uses only the configured final `X-Forwarded-For` hop and falls back to `unknown` when no address is available.

Defect 15: Existing indexes were not upgraded by `IF NOT EXISTS`

File: `apps/web/db/schema.sql`

`IF NOT EXISTS` leaves an old same-named index untouched on an existing database. The bootstrap now drops and recreates both cursor indexes:

```
DROP INDEX IF EXISTS notifications_user_created_idx;
CREATE INDEX notifications_user_created_idx
  ON notifications(user_id, created_at DESC, id DESC);

DROP INDEX IF EXISTS complaint_ai_analyses_complaint_idx;
CREATE INDEX complaint_ai_analyses_complaint_idx
  ON complaint_ai_analyses (complaint_id, created_at DESC, id DESC);
```

This ensures deployed databases receive the tie-breaker columns required by cursor ordering.

Defect 16: Password hashing held the reset transaction open

File: `apps/web/src/app/api/auth/reset/route.ts`

Scrypt hashing now runs before acquiring a database connection and beginning the reset transaction. Token consumption, password update, and session revocation remain atomic, while expensive CPU work no longer extends database transaction and row-lock duration.

Defect 17: Timestamp parameters caused real-database 500 responses

Files: `apps/web/src/app/api/rides/[id]/requests/route.ts`, `apps/web/src/lib/payment-settlement.ts`

Integration testing with PostgreSQL exposed untyped JavaScript `Date` parameters being used directly in timestamp/interval expressions. PostgreSQL could resolve the expression incorrectly and return 500 responses for seat requests and payment reminders. The affected parameters now use explicit `timestamptz` casts, and the regression is covered by `test/integration/bugs/timestamp-parameters.int.test.ts`.

Defect 18: Reusing an idempotency key replayed the wrong request

Files: `apps/web/src/lib/join-requests.ts`, `apps/web/src/app/api/rides/[id]/requests/route.ts`

Adversarial testing reused a key from one ride or pickup point on another request and received the first request as if it were a valid replay. The request now compares the original ride and payload identity, returns `idempotency_key_reused` for mismatches, and keeps the key-table uniqueness constraint as the database backstop. Regression coverage is in `test/integration/bugs/idempotency-key-reuse.int.test.ts`, including concurrent claims.

Remaining architectural observation

The schema remains a replayable bootstrap script rather than a versioned migration chain. This is a maintainability and deployment-risk observation, not a safe one-line defect fix; the compatibility ordering issues found during the audit were corrected without introducing a migration framework outside the scope of this deliverable.